

Prompting That Actually Works

The pro communication system — precision prompts, context stacking, negative constraints, and the senior engineer framing technique

There is a widely held belief that AI tools are only as smart as the model behind them. If the model is good, you get good results. If it is bad, you do not. The quality of your input, according to this belief, does not matter much.

This belief is wrong, and it is costing developers enormous amounts of time and frustration.

The difference in output quality between a developer who prompts well and a developer who prompts carelessly — using the same model, on the same task — can be an order of magnitude. We are not talking about marginal improvements. We are talking about the difference between code that works and code that does not. Between a session that takes fifteen minutes and one that takes two hours of back-and-forth. Between results you can ship and results you have to throw away.

This chapter teaches you the pro communication system. These are not tricks or hacks. They are the habits of thinking and expressing that professional developers who use Claude Code every day have converged on. Once you internalize them, they become second nature.

Why most prompts fail

Before we get to what works, it is worth understanding why most prompts fail. There are three failure modes that account for almost every disappointing Claude Code session.

Failure mode 1: vagueness

"Build me a login system." This prompt will produce something. But what it produces will probably not be what you wanted, because a login system can mean a hundred different things. What tech stack? What database? JWT or sessions? What validation rules? What error messages? Should it send a verification email? What does the UI look like?

Claude Code will make reasonable guesses, but they are guesses. The resulting code will need significant revision because it was built to satisfy an imagined requirement rather than your actual one.

Failure mode 2: missing context

"Add a caching layer to this function." Which caching strategy? Where is this function called, and how often? What is the acceptable staleness of the cached data? Is this a performance optimization or a cost reduction? What existing caching infrastructure exists in the project?

Without context, Claude Code cannot make good decisions. It fills the gaps with defaults, and defaults are rarely exactly what you need.

Failure mode 3: no constraints

"Refactor this module to be cleaner." Cleaner how? What is clean to you might not be clean to Claude Code. Should it reduce the number of functions? Reduce lines of code? Improve naming? Add comments? Reduce nesting? Split the module into multiple files?

Without constraints, Claude Code has no standard to hit. The result is code that is different from what you started with, but not necessarily better for your specific project and context.

Three Prompting Failure Modes

Vagueness

Example prompt:

`"Make this code better."`

Claude guesses.
Guesses are wrong.

Missing context

Example prompt:

`"Add error handling to this function."`

Defaults fill the gaps.
Defaults disappoint.

No constraints

Example prompt:

`"Build a dashboard for this data."`

No target =
no way to succeed.

The pro prompt structure

Professional Claude Code prompts share a consistent structure. You do not need to use every element every time, but understanding all of them — and knowing when each one matters — is what separates pro results from amateur ones.

A complete pro prompt has five components:

- Goal — what you want Claude Code to build or do
- Context — what already exists, what the constraints are, what the environment looks like
- Standards — how the output should be written (naming conventions, file structure, code style, libraries to use or avoid)
- Negative constraints — what Claude Code must NOT do, even if it seems like a good idea
- Success condition — how you will know the task is done correctly

You do not write these as a formal list. You write them as a natural paragraph or two. But you make sure all five are present.

BEFORE (beginner prompt)	AFTER (pro prompt)
Build me a user authentication system for my Node.js app.	Add JWT-based authentication to this Express app. The app already uses PostgreSQL with Knex — stay with that, do not add any new database libraries. Create a POST /auth/register route (email + password, bcrypt hash, store in users table) and a POST /auth/login route (returns a signed JWT with 24h expiry). Add a requireAuth middleware that validates the JWT and attaches req.user. Do not add refresh token logic — we will add that in a separate task. Do not modify any existing routes. Write integration tests for both auth routes using Jest and supertest. Tests should run with npm test.

Three Prompting Failure Modes

Vagueness

Example prompt:

"Make this code better."

Claude guesses.
Guesses are wrong.

Missing context

Example prompt:

"Add error handling to this function."

Defaults fill the gaps.
Defaults disappoint.

No constraints

Example prompt:

"Build a dashboard for this data."

No target =
no way to succeed.

The pro prompt structure

Professional Claude Code prompts share a consistent structure. You do not need to use every element every time, but understanding all of them — and knowing when each one matters — is what separates pro results from amateur ones.

A complete pro prompt has five components:

- Goal — what you want Claude Code to build or do
- Context — what already exists, what the constraints are, what the environment looks like
- Standards — how the output should be written (naming conventions, file structure, code style, libraries to use or avoid)
- Negative constraints — what Claude Code must NOT do, even if it seems like a good idea
- Success condition — how you will know the task is done correctly

You do not write these as a formal list. You write them as a natural paragraph or two. But you make sure all five are present.

BEFORE (beginner prompt)	AFTER (pro prompt)
Build me a user authentication system for my Node.js app.	Add JWT-based authentication to this Express app. The app already uses PostgreSQL with Knex — stay with that, do not add any new database libraries. Create a POST /auth/register route (email + password, bcrypt hash, store in users table) and a POST /auth/login route (returns a signed JWT with 24h expiry). Add a requireAuth middleware that validates the JWT and attaches req.user. Do not add refresh token logic — we will add that in a separate task. Do not modify any existing routes. Write integration tests for both auth routes using Jest and supertest. Tests should run with npm test.

16 | CLAUDE CODE PRO

The senior engineer framing technique

This is the single most effective prompting technique in this book. It takes thirty seconds to apply and consistently triples output quality.

When you write a prompt, frame yourself as a senior engineer briefing a capable junior developer who is new to this specific project. This framing is not just metaphorical — it structurally changes what information you include.

A senior engineer briefing a junior developer does not say "build me a login system." They say: here is the project, here is the tech stack, here is what already exists, here is what needs to be built, here are the constraints you need to respect, here is how we write code on this team, and here is how I will know you did it right.

When you write prompts with that briefing mindset, you naturally include goal, context, standards, constraints, and success conditions — all five components of the pro prompt structure — without having to think about them as a formal checklist.

PRACTICE EXERCISE

Before you write your next prompt, spend sixty seconds asking yourself: what would I tell a new team member before they started this task? Write that down. That is your prompt.

Precision prompts: being specific about the output

Precision in prompting means being specific not just about what you want, but about the exact form you want it in. This includes specifying:

File structure

Tell Claude Code where to create files, not just what to create. "Create a new file at src/middleware/auth.js" is more reliable than "create an auth middleware." If you have a specific folder structure you want preserved, say so.

Function signatures

If you know the interface you want, specify it. "The function should accept (userId: string, options: { includeDeleted?: boolean }) and return Promise<User | null>" gives Claude Code something concrete to build to. Without this, it invents an interface that may not match your needs.

Technology specificity

Do not say "add a cache." Say "use the Redis client already configured in src/lib/redis.js to cache the result of this function with a 10-minute TTL, keyed by userId." The more specific you are about which libraries, versions, and existing infrastructure to use, the more likely the output will integrate cleanly with what you already have.

Error handling style

Specify how you want errors handled — throw exceptions, return error objects, use a Result type, log to a specific logger. If you do not specify this, Claude Code will pick something reasonable that may not match your codebase's conventions.

PRECISION PROMPT ANATOMY

A well-structured prompt leaves no room for guesswork.

GOAL

Clear statement of what to build.

CONTEXT

Relevant details so Claude understands your environment.

STANDARDS

Specific requirements and technical expectations.

NEGATIVE CONSTRAINT

Explicit guardrails preventing mistakes.

SUCCESS CONDITION

A clear definition of "done."

```
PROMPT

Please create a function that paginates results.

Context: This is a Node.js project using TypeScript and Express.
The database is PostgreSQL using the pg library.
This function will be used in src/routes/users.ts.

Requirements:
• Accept page and limit as query parameters.
• Default page to 1 and limit to 20.
• Return { data, page, limit, total, totalPages }.
• Use parameterized queries to prevent SQL injection.

Do not: use any ORM, add global variables, or change existing
function signatures.

Success: Add unit tests in src/__tests__/pagination.test.ts that
cover happy path, page out of range, and invalid parameters.
Run the tests to confirm they pass.

Proceed when you're ready.
> 
```

Context stacking

Context stacking is the technique of building up Claude Code's understanding of your project across multiple messages before asking it to do the complex work.

Think of it this way: when a new developer joins your team, you do not hand them a ticket and walk away. You spend time explaining the project — the architecture, the patterns, the gotchas, the things that matter. That onboarding makes every subsequent conversation with that developer more productive, because they have the context they need to make good decisions.

You can do the same with Claude Code. In a new session, before you ask for anything complex, give Claude Code the context it needs:

```
> Before we start, let me give you context about this project.
This is a Node.js REST API using Express, TypeScript, and
PostgreSQL via Prisma. We follow a controller-service-repository
pattern. All errors are handled via a centralized error handler
in src/middleware/error.ts. We use Zod for request validation.
Tests are written with Jest. We never use callbacks — always
async/await. Understood?
```

Claude Code will confirm it understands. Now every subsequent request in that session will be answered with this context in mind. The code it generates will use Prisma, not raw SQL. It will use `async/await`, not callbacks. It will route errors through your error handler.

Context stacking is especially powerful when you put it in a `CLAUDE.md` file — a persistent configuration file that Claude Code reads automatically at the start of every session. We cover this in detail in Chapter 3.

Negative constraints: the most underused technique

Negative constraints — telling Claude Code what not to do — are the most underused prompting technique and one of the most powerful.

By default, Claude Code will make what it considers to be reasonable choices. But "reasonable" is a broad category, and its reasonable choices may conflict with your architecture, your existing conventions, or specific decisions you have already made.

Negative constraints prevent these conflicts before they happen. Some examples of effective negative constraints:

- "Do not add any new npm packages — use only what is already in package.json."
- "Do not modify any existing test files — only add new ones."
- "Do not change the function signatures of any existing functions."
- "Do not use class components — this project uses only functional React components with hooks."
- "Do not add console.log statements — use the logger at src/lib/logger.ts."
- "Do not make any changes outside of the src/features/billing/ directory."

Notice that most of these constraints are things Claude Code would do in the name of being helpful — add a package that makes the task easier, add a console.log for debugging, touch a related file to make things consistent. Without explicit constraints, these seemingly helpful additions create problems: broken dependency policies, noisy production logs, accidental regressions in untouched code.

❑ COMMON MISTAKE

Developers often add negative constraints only after Claude Code has already done the wrong thing. Add them upfront. The cost of specifying a constraint you did not need is zero. The cost of missing a constraint you did need is a broken build and a frustrating debugging session.

Multi-turn conversations: how to iterate effectively

Claude Code sessions are conversations, not single prompts. Understanding how to iterate effectively across multiple turns is what separates sessions that produce polished results from sessions that go in circles.

Build in layers

Do not try to specify everything in a single prompt. Start with the core functionality, confirm it works, then layer in additional requirements. This approach is more reliable and gives you meaningful checkpoints along the way.

```
> Turn 1: Build the basic user registration endpoint.  
Email, password, store in database. Return 201 on success.  
  
> Turn 2: Add input validation. Email must be valid format,  
password minimum 8 characters, at least one number.  
Return 400 with a descriptive message on validation failure.  
  
> Turn 3: Add rate limiting — maximum 5 registration attempts  
per IP address per hour. Use the existing Redis instance.
```

Be specific about what to change

When asking Claude Code to revise something, be precise about what specifically needs to change. "Make the error messages better" will produce unpredictable results. "Change the error message for duplicate email from 'Conflict' to 'An account with this email address already exists. Please log in or use a different email.'" will produce exactly what you want.

Reference specific files and functions

In a revision request, always reference the specific file and function you are talking about. This eliminates ambiguity and prevents Claude Code from modifying something other than what you intended.

```
> In src/controllers/auth.ts, in the registerUser function,
  change the bcrypt salt rounds from 10 to 12.
```

30 copy-paste prompt templates

The following templates are organized by task. Replace the bracketed placeholders with your specifics. Each template includes the five components of the pro prompt structure.

Code generation

Template G1 – Build a new feature

```
Add [FEATURE NAME] to [FILE/MODULE].
Context: [describe what already exists and how this fits in].
Tech stack: [list relevant libraries/frameworks to use].
Requirements: [list specific behaviors].
Do not: [list things to avoid].
Done when: [describe the success condition, e.g., tests pass].
```

Template G2 – Create a new API endpoint

```
Add a [METHOD] [/path] endpoint to [ROUTER FILE].
It should [describe what it does].
Input: [describe request body/params/query].
Output: [describe response format and status codes].
Validation: [describe required validation].
Use [existing middleware/services] – do not create new ones.
Write a test for this endpoint in [TEST FILE].
```

Debugging

Template D1 – Fix a specific bug

```
In [FILE], the [FUNCTION NAME] function is producing
incorrect output. When I call it with [INPUT], I expect
[EXPECTED OUTPUT] but I get [ACTUAL OUTPUT].
Do not change the function's external interface.
Add a test that would have caught this bug.
```

Template D2 – Debug from an error message

```
I am getting this error: [PASTE ERROR MESSAGE].
It occurs when [describe what triggers it].
Relevant files: [list files involved].
```


Fix the root cause. Do not suppress the error.
Explain briefly what caused it.

Testing

Template T1 – Write tests for existing code

Write tests for [FILE/FUNCTION] using [TESTING FRAMEWORK].
Cover: happy path, all error cases, edge cases for
[list specific scenarios to include].
Mock: [list dependencies to mock].
Do not test implementation details – test behavior.
Tests go in [TEST FILE PATH].

Refactoring

Template R1 – Refactor for clarity

Refactor [FILE/FUNCTION] to improve [readability/performance/maintainability]. Specific goals: [describe what should improve].
Constraints: do not change public interfaces, do not change behavior, all existing tests must still pass.
Add a comment above any non-obvious logic after refactoring.

You will find the complete library of 150 prompt templates — organized by language, framework, and task type — in the downloadable resources at [github.com/\[your-handle\]/claude-code-pro-resources](https://github.com/[your-handle]/claude-code-pro-resources).

CHAPTER 2 SUMMARY

Great prompts have five components: goal, context, standards, negative constraints, and success condition. The senior engineer framing technique — briefing Claude Code as you would brief a capable junior developer — is the fastest way to include all five naturally. Context stacking builds Claude Code's understanding before you ask for complex work. Negative constraints prevent the reasonable-but-wrong choices that derail sessions. Multi-turn conversations work best when you build in layers. Chapter 3 takes everything you have learned about communicating with Claude Code and makes it permanent — through a project configuration system that means you never have to repeat the same context twice.